ЗЕРТХАНАЛЫҚ ЖҰМЫСТАР ЕСЕБІ

StayBook

Қонақүй нөмірлерін брондау ақпараттық жүйесі

№1 · №2 · №3 · №4

Білім алушы: _____

Тобы: _____

Оқытушы: _____

Тапсыру күні: _____

Python · FastAPI · SQLModel · SQLite · Streamlit

Есептің құрамы

- Жоба туралы
- №1 зертханалық жұмыс
- №2 зертханалық жұмыс
- №3 зертханалық жұмыс
- №4 зертханалық жұмыс
- Қорғауға дайындық

Бұл есеп бір ақпараттық жүйені жобалаудан бастап, дерекқорын, REST API және толық CRUD әрекеттерін іске асыруға дейінгі төрт зертханалық жұмысты біріктіреді.

Титул бетіндегі білім алушы, топ, оқытушы және тапсыру күні жолдары тапсырар алдында толтырылады.

№1–4 зертханалық жұмыстар бойынша есеп

Пән: қолданбалы ақпараттық жүйелерді әзірлеу **Жоба:** StayBook — қонақүй нөмірлерін брондау жүйесі **Орындаған студент:** _____ **Топ:** _____ **Тексерген оқытушы:** _____ **Тапсыру күні:** _____

Жобаның мақсаты

Қонақүйдің нөмірлері, олардың санаттары, қонақтар және брондаулар туралы мәліметті сақтайтын ақпараттық жүйе құру. Python, SQLModel, SQLite және FastAPI көмегімен деректерді қосу, оқу, өзгерту, жою және сүзгілеу әрекеттерін біріктіру.

Қолданылған құралдар

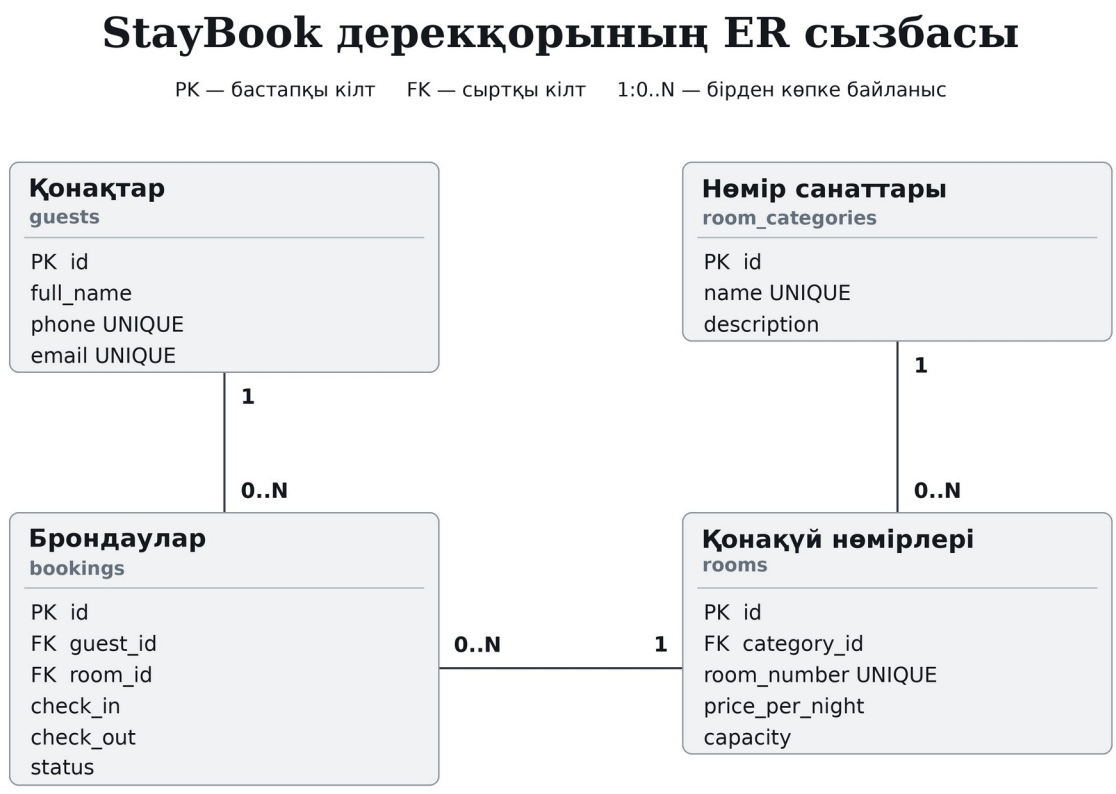
Құрал	Міндеті
Python	Қолданбаның негізгі бағдарламалау тілі
FastAPI	HTTP маршруттары, сұрауларды өңдеу, OpenAPI
SQLModel / SQLAlchemy	ORM модельдері, байланыстар, транзакциялар
SQLite	Жергілікті реляциялық дерекқор
Pydantic	Кіріс деректерін және жауап схемаларын тексеру
Uvicorn	ASGI сервері
Streamlit	API-мен жұмыс істейтін қосымша оқу интерфейсі
Git / GitHub	Нұсқалар тарихы және қашықтағы репозиторий
Pytest / HTTPX	API мен бизнес шарттарын автоматты тексеру

№1. Ақпараттық жүйені жобалау

Пәндік сала ретінде қонақүй нөмірлерін брондау таңдалды. Басты әрекеттер: қонақты тіркеу, санат пен нөмірді қосу, баға және сыйымдылықты сақтау, келу/кету мерзіміне брондау жасау.

Төрт кесте жобаланды:

1. `guests` — қонақтың аты-жөні, бірегей телефон нөмірі және электрондық поштасы.
2. `room_categories` — нөмір санаты және оның сипаттамасы.
3. `rooms` — санатқа тиесілі нөмір, түндік баға және сыйымдылық.
4. `bookings` — қонақ пен нөмірге сілтеме, келу/кету күндері және мәртебе.



StayBook дерекқорының ER сызбасы

1-сурет — ER-диаграмма. Бастапқы кілт `PK`, сыртқы кілт `FK` арқылы белгіленді. Бір санатқа бірнеше нөмір, бір қонаққа бірнеше брондау және бір нөмірге әртүрлі күндердегі бірнеше брондау сәйкес келеді.

Жоба сервер, дерекқор, интерфейс, тест және құжаттама бөлімдеріне бөлінді. Тәуелділіктер файлдары мен `.gitignore` дайындалды. Виртуалды орта және IDE-мен жұмыс істеу командалары README мен №1 жұмыс нұсқаулығында берілді. Cursor/VS Code орнатуды және жобаны студенттің өз компьютерінде іске қосуды студент жергілікті орындайды.

№2. Реляциялық база және ORM

ER-сызба `app/models.py` файлындағы SQLModel кластарына көшірілді. Модельдер бастапқы және сыртқы кілттерді, бірегей мәндерді және `Relationship` байланыстарын сипаттайды. Баға үшін `Decimal`, брондау күндері үшін `date` қолданылады.

`app/db.py` дерекқор engine-ін, кестелерді дайындайтын `init_db()` және сұрауға сессия беретін `get_session()` функцияларын қамтиды. SQLite сыртқы кілттерді тексереді. Базаның әдепкі файлы — `database.db`.

`seed.py` скрипті бастапқы деректерді енгізуге арналған. Бірдей деректерді қайталап қоспау үшін табиғи бірегей мәндер тексеріледі. Скриптті іске қосу:

```
python seed.py
```

`database.db`, виртуалды орта, кэштер мен құпия конфигурациялар Git-ке қосылмайды. Қайта қалпына келтіруге қажет модельдер мен seed коды репозиторийде сақталады.

№3. FastAPI сервері

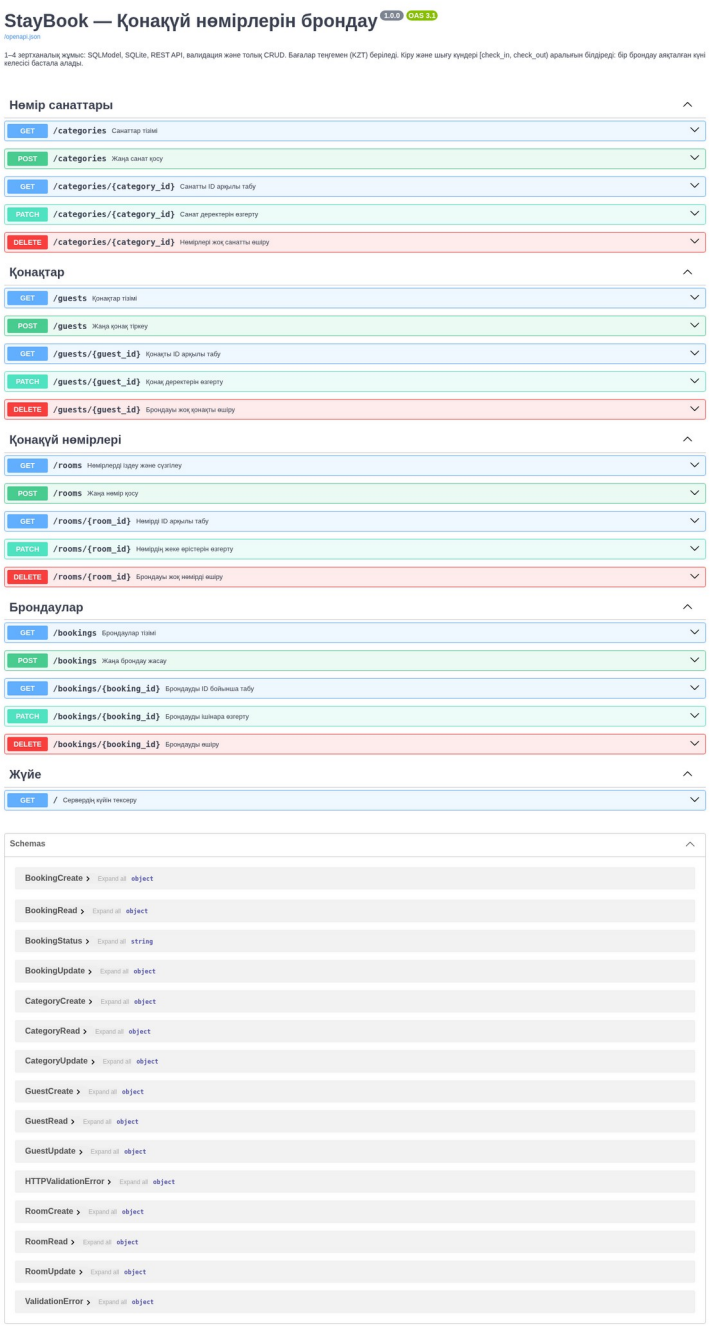
Қолданбаның кіру нүктесі — `app/main.py`. Серверді іске қосу:

```
uvicorn app.main:app --reload
```

Жүйеде нөмірлер, қонақтар, санаттар және брондаулар үшін маршруттар қарастырылған. №3 жұмыстың негізгі әрекеттері — нөмірлер тізімін оқу, ID арқылы табу және жаңа нөмір қосу. Әр сұрау `Depends(get_session)` арқылы дерекқор сессиясын алады.

Сұрау схемалары ORM кестелерінен бөлінген. Нөмір қосқанда баға, бос белгі, санаттың бар болуы және сыйымдылық тексеріледі. Сәтті жасау `201`, жарамсыз дерек `422`, табылмаған ID `404`, қайталанатын бірегей мән `409` мәртебесімен беріледі.

Swagger UI мекенжайы — <http://127.0.0.1:8000/docs>. Бұл құжаттама маршруттар мен схемалардан автоматты жасалады.



StayBook Swagger UI

2-сурет — API-дің Swagger UI көрінісі. Нақты орындалған тексерулер бөлек тексеру есебінде беріледі.

№4. Толық CRUD және бизнес ережелері

`PATCH /rooms/{room_id}` жіберілген өрістерді ғана өзгертеді. `model_dump(exclude_unset=True)` жіберілмеген өрістерді сақтайды, ал міндетті өрістерге нақты `null` жіберу валидациямен қабылданбайды.

`DELETE /rooms/{room_id}` жойылатын нөмірдің бар болуын және байланысты брондауларын тексереді. Сәтті жою `204` бос жауабын береді. Жазба жоқ болса `404`, байланысты деректі бұзатын әрекет болса `409` қайтарылады.

Тізімді сүзгілеу query параметрлерімен беріледі:

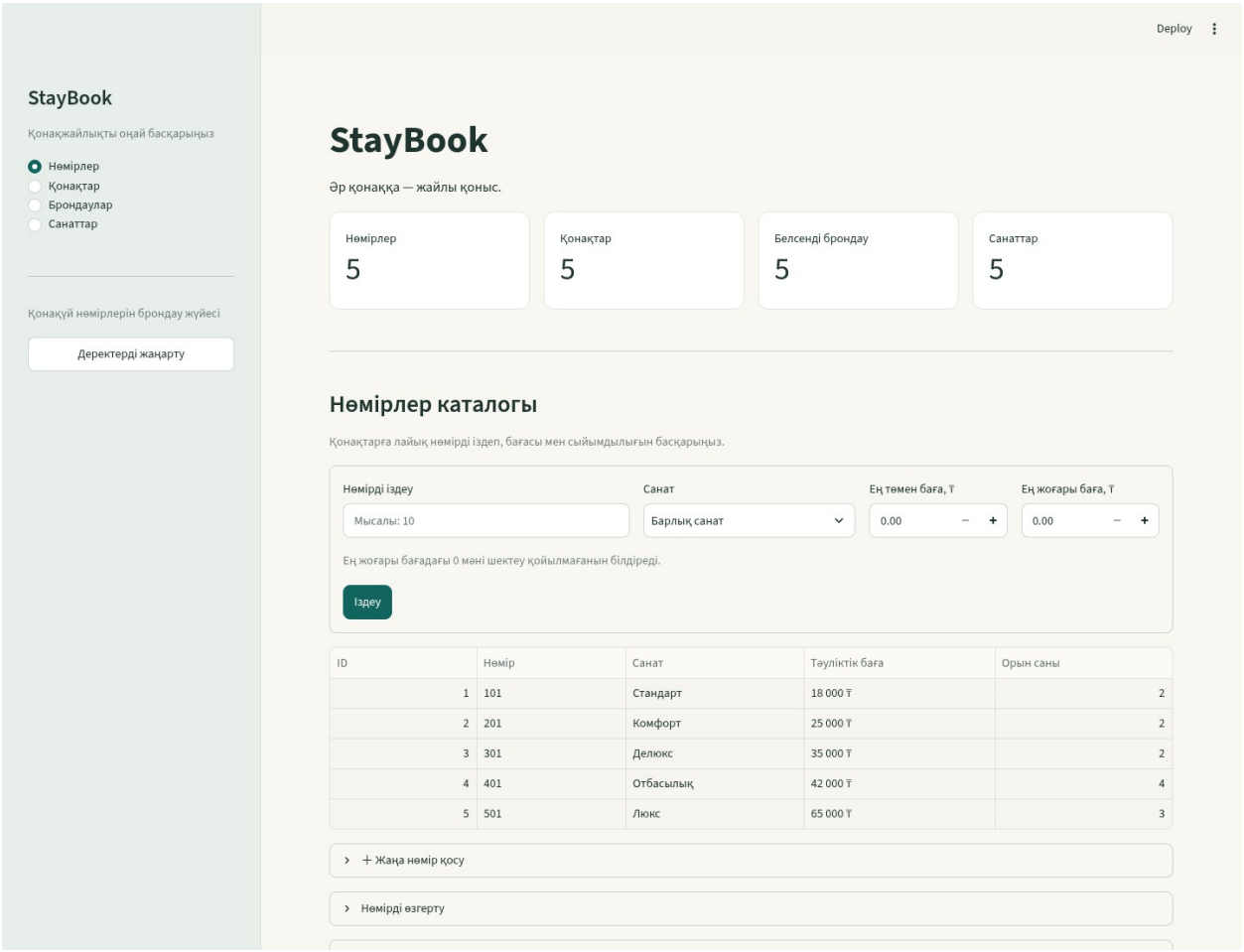
```
GET /rooms?search=10&min_price=10000&max_price=30000&category_id=1
```

SQL сұраулары SQLModel конструкцияларымен жасалады. Іздеу жолы SQL мәтініне тікелей жалғанбайды. Төменгі және жоғарғы баға шектері тексеріледі.

Брондауда келу күні кету күнінен ерте болуы керек. Қонақ пен нөмір дерекқорда болуы тиіс. Бір нөмірге қабаттасатын, күшін жоймаған брондау қабылданбайды. Кету күні келесі қонақтың келу күнімен сәйкес келуі мүмкін.

Қосымша Streamlit интерфейсі серверге HTTP арқылы қатынауға арналған:

```
streamlit run frontend/app.py
```



StayBook интерфейсі

3-сурет — қосымша қолданушы интерфейсі. API бөлек терминалда іске қосылады.

Негізгі тексеру жағдайлары

Бұл кесте күтілетін мінез-құлықты сипаттайды. Нақты тест саны мен іске қосу нәтижелері verification.md файлында беріледі.

Әрекет	Күтілетін нәтиже
Нөмірлер тізімін алу	`200`, тізім
Жарамды нөмір қосу	`201`, жасалған жазба
Теріс бағамен нөмір қосу	`422`
Тек бағаны PATCH арқылы өзгерту	`200`, басқа өрістер сақталады
Атау бөлігі және баға бойынша сүзгілеу	Шартқа сәйкес жазбалар
Брондаусыз нөмірді жою	`204`, бос жауап

Әрекет	Күтілетін нәтиже
Жойылған ID-ге қайта жүгіну	`404`
Бірегей нөмір белгісін қайталау	`409`
Мерзімі қабаттасатын брондау	`409`
Seed скриптін қайта орындау	Қайталанатын бастапқы жазбалар қосылмайды

Git тарихын ұйымдастыру

Төрт жұмысты кезеңдерге бөлетін коммит хабарламалары:

```
feat: initial project structure with AI assist
feat: add SQLModel ORM models and database seed script
feat: implement FastAPI endpoints for GET and POST with validation
feat: complete CRUD operations with PATCH, DELETE and query filtering
```

Нақты коммиттерді `git log --oneline` командасымен көруге болады. GitHub репозиторийі бастапқы код пен құжаттарды сақтайды; серверді автоматты іске қоспайды.

Қорғауға дайындық

Студент ER-диаграмманы, кестелердің байланысын және әр негізгі файлдың міндетін түсіндіреді. Swagger арқылы GET, POST, PATCH, DELETE және қате сұрауларды көрсетеді. Жиырма бақылау сұрағының қысқа жауаптары defense_questions.md файлында берілген.

Зертханалық жұмыс №1

Тақырып: ақпараттық жүйені жобалау, ИИ көмегімен әзірлеу ортасын және нұсқаларды басқаруды дайындау.

Жоба: StayBook — қонақүй нөмірлерін брондау жүйесі.

Мақсат

Пәндік саланы кестелерге бөлу, ER-диаграмма құру, Python жобасының құрылымын дайындау және Git арқылы өзгерістерді сақтау тәртібін түсіну.

1. Пәндік саланы анықтау

Қонақүй қызметкері қонақтарды, нөмір санаттарын, нөмірлердің бағасы мен сыйымдылығын және брондауларды есепке алады. Нөмір бірнеше рет брондалуы мүмкін, бірақ күшін жоймаған брондаулардың күндері қабаттаспайды.

Төрт негізгі мән таңдалды:

- `guests` — қонақтар;
- `room_categories` — нөмір санаттары;
- `rooms` — қонақүй нөмірлері;
- `bookings` — брондаулар.

Байланыстар: `room_categories → rooms`, `guests → bookings`, `rooms → bookings`. Барлығы — «бірден көпке»: `1 — 0..N`.

2. ER-диаграмма

Сызбадағы кестелердің қазақша тақырыбы, ағылшынша техникалық атауы және өрістері көрсетілген. `PK id` әр жазбаны анықтайды. `FK category_id`, `FK guest_id`, `FK room_id` кестелерді байланыстырады.

Тапсырылатын файлдар: `er_diagram.png`, масштабталатын `er_diagram.svg` және draw.io ішінде өңделетін `er_diagram.drawio`.

3. Git репозиторийі

Қашықтағы ашық репозиторий атауы: `applied-is-project-2026`. Негізгі сипаттама `README.md` файлында беріледі, Python ортасы мен уақытша файлдар `.gitignore` арқылы алынып тасталады.

Жергілікті жүктеу:

```
git clone <репозиторийдің_HTTPS_сілтемесі>
cd applied-is-project-2026
```

Жобаның репозиторийі әлі жасалмаған жағдайда GitHub-та **New repository → Public**, атауын енгізу және README мен Python `.gitignore` қосу қадамдары орындалады.

4. Жергілікті IDE және орта

Студент өз компьютеріне Cursor немесе VS Code және ИИ көмекшісін орнатады, жоба папкасын ашады. IDE орнату студенттің компьютерінде орындалатын қадам; репозиторийдегі код бұл орнатуды алмастырмайды.

```
python -m venv venv
```

Windows PowerShell: `.\venv\Scripts\Activate.ps1`. Linux/macOS: `source venv/bin/activate`.

```
python -m pip install -r requirements-dev.txt
```

ИИ-ге беруге болатын сұрау үлгісі:

> *Python тілінде қонақүй нөмірлерін брондау жүйесін жасаймын. FastAPI сервері, SQLModel дерекқоры және Streamlit интерфейсі үшін түсінікті папка құрылымын ұсын. fastapi, uvicorn, sqlmodel, streamlit тәуелділіктерін көрсет. Әр файлдың міндетін түсіндір.*

Жобада `app/`, `app/routers/`, `frontend/`, `tests/`, `docs/`, `scripts/` бөлімдері қарастырылған. Толық қызметтер кейінгі зертханаларда осы құрылымға қосылады.

5. Өзгерістерді сақтау

```
git add README.md .gitignore requirements.txt requirements-dev.txt app frontend docs scripts
git commit -m "feat: initial project structure with AI assist"
git push origin main
```

Командалар орындау тәртібін көрсетеді. Алдын ала жасалған коммиттерді қайталау қажет емес; нақты тарих `git log --oneline` арқылы қаралады.

Қорғауда көрсету

ER-диаграммадағы үш байланысты түсіндіру, виртуалды ортаны белсендіру, `.gitignore` және README файлын ашу. Қорытынды бақылау сұрақтарының жауаптары осында.

Зертханалық жұмыс №2

Тақырып: реляциялық дерекқор және SQLModel ORM модельдері.

Мақсат

Бірінші жұмыстағы ER-сызбаны Python кластарына көшіру, кілттер мен объектілік байланыстарды баптау және SQLite базасына мысал жазбалар енгізу.

1. Модельдер

`app/models.py` файлында төрт кесте анықталады:

Модель	Кесте	Негізгі өрістер
Қонақ	`guests`	`id`, `full_name`, `phone`, `email`
Нөмір санаты	`room_categories`	`id`, `name`, `description`
Қонақүй нөмірі	`rooms`	`id`, `category_id`, `room_number`, `price_per_night`, `capacity`
Брондау	`bookings`	`id`, `guest_id`, `room_id`, `check_in`, `check_out`, `status`

`id` — бастапқы кілт. `category_id`, `guest_id`, `room_id` — сыртқы кілттер. Қонақтың телефоны мен email-і, санат атауы және нөмір белгісі қайталанбауы тиіс. Ақша `Decimal` негізіндегі сандық өріспен беріледі, күндер `date` түрінде сақталады.

`Relationship` арқылы бір санаттың нөмірлеріне, бір қонақтың брондауларына және бір нөмірдің брондауларына Python объектілері ретінде қатынауға болады. Бұл өрістердің өзі SQL кестесіне бөлек баған болып қосылмайды.

ИИ сұрауының үлгісі:

> *ER-сызбаға сәйкес Guest, RoomCategory, Room, Booking SQLModel модельдерін жаса. Бастапқы және сыртқы кілттерді, бірегей өрістерді, бірден көпке Relationship байланыстарын көрсет. Бағаға Decimal, брондау күндеріне date қолдан. Кесте атаулары guests, room_categories, rooms, bookings болсын.*

2. Дерекқорды қосу

`app/db.py` файлындағы `engine` дерекқорға қосылымдарды басқарады. Әдепкі URL:

```
sqlite:///./database.db
```

`init_db()` функциясы `SQLModel.metadata.create_all(engine)` көмегімен жоқ кестелерді жасайды. SQLite үшін сыртқы кілттерді тексеру қосылады. `DATABASE_URL` арқылы бөлек тестілік база таңдауға болады.

Кестелер схемасын өзгерткенде `create_all()` бұрыннан бар бағандарды автоматты көшірмейді. Бұл оқу жобасында бастапқы схеманы қайта жасау үшін бөлек жаңа база қолдануға болады; құнды деректері бар базада миграция қажет.

3. Бастапқы деректер

Жобаның түбірінде:

```
python seed.py
```

Скрипт алдымен кестелерді дайындайды, содан соң әр кестеге 5 тестілік жазба енгізеді. Жазбалар жалған мысал деректер ретінде қолданылады. Қайта орындау жазбаларды орынсыз көбейтпеуі тиіс.

SQL командаларын көру, Linux/macOS:

```
SQL_ECHO=1 python seed.py
```

Windows PowerShell:

```
$env:SQL_ECHO = "1"
python seed.py
Remove-Item Env:SQL_ECHO
```

Логта кестелерді тексеру/жасау және алғашқы іске қосуда деректерді енгізу сұраулары көрінеді. Кесте бұрын жасалған болса, әр іске қосуда `CREATE TABLE` қайталануы міндетті емес.

4. Git-те сақтау

`.gitignore` ішінде кемінде `venv/`, `.venv/`, `*.db`, `.env`, `__pycache__/` болуы керек. Код сақталады, ал база мен виртуалды орта қайта құрылады.

```
git add app/models.py app/db.py seed.py .gitignore
git commit -m "feat: add SQLModel ORM models and database seed script"
git push origin main
```

Қорғауда көрсету

`python seed.py` командасын орындау, сыртқы кілттерді және `Relationship` мағынасын түсіндіру. Бір кестеге қатысты Python класын ER-сызбадағы блокпен салыстыру. Бақылау сұрақтарының жауаптары осында.

Зертханалық жұмыс №3

Тақырып: FastAPI арқылы серверлік REST API жасау.

Мақсат

Дерекқордағы мәліметті HTTP арқылы оқу және қосу, сұрауларды валидациялау, әр сұрауға бөлек сессия беру және Swagger UI арқылы API-ді көрсету.

1. Қолданбаны іске қосу

Виртуалды орта белсенді болуы керек. Алдымен мысал деректерді дайындаңыз:

```
python seed.py
uvicorn app.main:app --reload
```

`app/main.py` FastAPI қолданбасын жасайды, lifespan кезінде `init_db()` шақырады және роутерлерді қосады. `GET /` сервердің күйін хабарлайды.

Дұрыс мекенжайлар:

- сервер: <http://127.0.0.1:8000/>;
- Swagger: <http://127.0.0.1:8000/docs>.

Бастапқы нұсқаулықтағы `http://127.0.0` толық емес; стандартты uvicorn порты — `8000`, Swagger жолы — `/docs`.

2. Сессияны тәуелділік арқылы алу

`app/db.py` ішіндегі `get_session()` функциясы сессияны ашады, `yield` арқылы маршрутқа береді және сұраудан кейін жабады. Маршрутта `Depends(get_session)` қолданылады. Барлық сұрауға бір ортақ ашық `Session` объектісін пайдалану дұрыс емес: транзакциялар мен сұраулар араласып кетуі мүмкін.

3. GET маршруттары

`app/routers/` бөлімінде:

- `GET /rooms` — нөмірлер тізімі;
- `GET /rooms/{room_id}` — ID бойынша нөмір;
- жоқ ID үшін `404 Not Found`.

Қосымша `/guests`, `/categories`, `/bookings` маршруттары байланысты деректерді дайындауға және көрсетуге арналған.

ИИ сұрауының үлгісі:

> *FastAPI және SQLModel көмегімен нөмірлер тізімін және ID бойынша бір нөмірді қайтаратын GET маршруттарын жаз. Сессияны Depends(get_session) арқылы ал. Жазба табылмаса HTTPException 404 қайтар. Жауап схемасын ORM моделінен бөлек анықта.*

4. POST және валидация

`app/schemas.py` ішінде клиенттен келетін дерекке арналған құру схемалары және жауап схемалары бөлек беріледі. Клиент бастапқы кілтті өз бетімен орнатпайды.

`POST /rooms` мысалы:

```
{
  "category_id": 1,
```

```
  "room_number": "DEMO-601",
  "price_per_night": "22000.00",
  "capacity": 2
}
```

`category_id=1` базада болуы керек. Сәтті қосылған жазба `201 Created` күйімен қайтады. Теріс баға, бос атау/белгі немесе жарамсыз сыйымдылық `422` қатесін беруі керек. Бірегей нөмір белгісінің қайталануы — `409 Conflict`.

ИИ сұрауының үлгісі:

> *Нөмір қосатын POST /rooms маршрутын жаса. Баға теріс болмасын, нөмір белгісі бос болмасын, сыйымдылық кемінде бір болсын. Санат бар екенін тексер. Жазбаны session.add/commit/refresh арқылы сақтап, 201 мәртебесін қайтар. Қате транзакцияда rollback жаса.*

5. Swagger арқылы көрсету

1. `/docs` ашып, `GET /rooms → Try it out → Execute` таңдаңыз.
2. `GET /rooms/{room_id}` ішінде тізімдегі ID-ді көрсетіңіз.
3. Теріс бағамен жаңа нөмір қосуға тырысыңыз: күтілетін код — `422`.
4. Жарамды дерек жіберіңіз: `201` және жасалған нөмірдің ID-і қайтуы керек.
5. Жоқ ID-ге GET жіберіңіз: күтілетін код — `404`.

Қолданбаның нақты түсірілген Swagger суреті: `docs/screenshots/swagger.png`. Тексерудің нақты қорытындылары `docs/verification.md` файлында сақталады.

Git-те сақтау

```
git add app/
git commit -m "feat: implement FastAPI endpoints for GET and POST with validation"
git push origin main
```

Бақылау сұрақтарының жауаптары осында.

Зертханалық жұмыс №4

Тақырып: толық CRUD, ішінара өзгерту, жою, іздеу және сүзгілеу.

Мақсат

№3 жұмыстағы оқу және қосу әрекеттерін толықтырып, деректерді қауіпсіз өзгерту мен жоюды, query параметрлерімен іздеуді және қате жағдайларды өңдеуді жүзеге асыру.

1. PATCH схемасы

Жаңарту схемасындағы өрістерді өткізіп жіберуге болады. `model_dump(exclude_unset=True)` клиент жібермеген өрістерді жаңартуға қоспайды. Сондықтан тек бағаны жібергенде нөмір белгісі, санаты мен сыйымдылығы сақталады.

`Optional` немесе әдепкі `None` мәні деректер базасындағы міндетті өрісті автоматты қорғамайды. Клиент міндетті өріске нақты `null` жіберсе, схема оны валидация кезінде қабылдамауы тиіс. Бұл «өрісті жібермеу» мен «өріске null жіберу» арасындағы айырма.

2. Нөмірді өзгерту

`PATCH /rooms/{room_id}` денесі:

```
{
  "price_per_night": "24500.00"
}
```

Маршрут ID бойынша жазбаны іздейді. Жоқ болса `404` қайтарады. Бар болса жіберілген өрістерді ғана жаңартып, өзгерісті базаға сақтайды. `GET` арқылы бағаның өзгергенін және қалған өрістердің сақталғанын салыстыруға болады.

ИИ сұрауының үлгісі:

> *PATCH /rooms/{room_id} жаса. Бастапқы жазба табылмаса 404 бер. Тек жіберілген өрістерді model_dump(exclude_unset=True) және sqlmodel_update арқылы өзгерт. Теріс баға мен міндетті өрістерге null қабылдама. Санатты ауыстырғанда сыртқы кілттің бар екенін тексер.*

3. Жою

`DELETE /rooms/{room_id}`:

- жоқ ID — `404`;
- байланысқан брондауы бар нөмір — `409`;
- жоюға болатын нөмір — `204 No Content`.

`204` жауабының денесі болмайды. Нөмірдің брондаулары бар кезде оны бірден жоймау байланысқан деректердің жоғалуын болдырмайды.

Көрсету үшін алдымен брондаусыз жаңа нөмір қосып, соны жойыңыз. Seed деректеріндегі нөмірдің брондауы болуы мүмкін.

4. Іздеу және баға бойынша сүзгілеу

`GET /rooms` параметрлері:

Параметр	Қызметі
`search`	Нөмір белгісінің бір бөлігін іздеу
`min_price`	Ең төмен түндік баға
`max_price`	Ең жоғары түндік баға
`category_id`	Бір санаттағы нөмірлер

Мысал:

```
http://127.0.0.1:8000/rooms?search=10&min_price=10000&max_price=30000
```

Фильтрлер `select(...).where(...)` түріндегі параметрленген SQLModel сұраулары арқылы орындалады. Пайдаланушы енгізген мәтін SQL жолына тікелей жалғанбайды. Теріс баға шегі немесе төменгі шектің жоғарғы шектен үлкен болуы қате сұрау ретінде тексеріледі.

5. Брондау бизнес ережелері

Брондауды жасау кезінде қонақ пен нөмірдің бар болуы тексеріледі. `check_out` күні `check_in` күнінен кейін болуы керек. Бір нөмірдің күшін жоймаған брондаулары `[check_in, check_out)` аралықтарында қабаттаспайды: бір қонақ шыққан күні келесі қонақ келе алады. Қайшылыққа `409` қайтарылады. `cancelled` брондау нөмірді сол мерзімге босатады.

6. Қолмен тексеру сценарийі

1. `GET /rooms` нәтижесін және бір нөмірдің бастапқы өрістерін қараңыз.
2. Іздеу мен баға шектерін беріп, нәтижеге тек сәйкес жазбалардың кіретінін тексеріңіз.
3. Жаңа нөмір құрып, оның тек бағасын PATCH арқылы өзгертіңіз.
4. Сол нөмірді GET арқылы қайта алып, қалған өрістерді салыстырыңыз.
5. Нөмірді жойыңыз; сол ID-ге PATCH қайталағанда `404` күтіледі.
6. Бір нөмірге қабаттасатын екі брондау жасаңыз: екіншісі `409` беруі тиіс.

Автоматты тексеру:

```
python -m pytest -q
```

Нақты орындалу нәтижелері тексеру есебінде беріледі. №4 жұмыстың тақырыбында PUT аталғанымен, нұсқаулықтың орындалу қадамдары мен бағалау критерийлері PATCH-ті талап етеді.

Git-те сақтау

```
git add app/ tests/ docs/
git commit -m "feat: complete CRUD operations with PATCH, DELETE and query filtering"
git push origin main
```

Қорғауда PATCH пен PUT айырмасын, `exclude_unset=True` қызметін және жоқ ID-ге жауапты түсіндіру керек. Бақылау сұрақтарының жауаптары осында.

Қорғауға арналған бақылау сұрақтары

Жауаптар бастапқы нұсқаулықтағы №1–4 жұмыстардың 20 бақылау сұрағына сәйкес реттелді. Мысалдар StayBook жобасына бейімделді.

Зертханалық жұмыс №1

1. Python жобасында виртуалды орта не үшін керек? Оны құрмасақ не болады?

`venv` бір жобаның Python пакеттерін басқа жобалардың пакеттерінен бөледі. StayBook тәуелділіктері сол ортаға орнатылады. Онсыз жүйелік Python ортасында кітапхана нұсқалары қайшы келуі, бір жобаны жаңарту екіншісін бұзуы мүмкін. Виртуалды орта Python нұсқасын өзі орнатпайды, ол таңдалған Python интерпретаторының негізінде жасалады.

2. `.gitignore` не үшін қажет? Неліктен оған `venv` және парольдері бар файлдар қосылады?

`.gitignore` Git-ке әлі тіркелмеген қандай файлдарды бақылауға алмау керегін көрсетеді. Виртуалды орта үлкен әрі басқа компьютерде қайта орнатылады, ал құпия конфигурация жарияланбауы тиіс. Жобаға `venv/`, `.venv/`, `.env`, `*.db`, `__pycache__/` кірмейді. Бұрын Git-ке қосылған файлды `.gitignore`-ға жазу оны тарихтан автоматты алып тастамайды.

3. Cursor немесе ИИ плагині қандай қызмет атқарады? ИИ кодын тексермей көшірудің тәуекелі қандай?

ИИ көмекшісі жоба құрылымын ұсынуға, модель мен API кодын жазуға, қатені түсіндіруге және тест дайындауға көмектеседі. Әзірлеуші берілген кодтың логикасын тексереді, қолданылған кітапхананың құжатымен салыстырады және іске қосады. Тексерілмеген код ескірген API-ді, қате байланыстарды, қауіпсіз емес сұрауларды немесе орындалмайтын функцияларды қамтуы мүмкін.

4. Git тармағы деген не? Неліктен командалар негізгі тармаққа тікелей жазуды шектейді?

Тармақ — коммиттер тарихындағы бөлек бағытты көрсететін сілтеме. `git branch feature/room-filters` жаңа тармақ жасайды; `git switch -c feature/room-filters` оны жасап, соған ауыстырады. Командаларда `main` жиі қорғалады: өзгеріс бөлек тармақта жазылып, pull request пен тексерулерден кейін біріктіріледі. Бұл барлық жобадағы Git-тің міндетті тыйымы емес, репозиторийдің жұмыс тәртібі.

5. `git commit` пен `git push` айырмасы неде? Өзгерістер қайда сақталады?

`git commit` индекске қосылған өзгерістердің нұсқасын жергілікті `.git` тарихына сақтайды. `git push` жергілікті коммиттерді GitHub сияқты қашықтағы репозиторийге жібереді. Тек commit жасалса, өзгеріс басқа компьютерлерде әлі көрінбейді. `git add` файлдарды келесі коммитке дайындайды.

Зертханалық жұмыс №2

1. ORM деген не және ол қандай мәселені шешеді?

ORM — Object-Relational Mapping, Python объектілерін реляциялық кестелермен сәйкестендіру. SQLModel ішінде `Room` класының объектісі `rooms` кестесінің жазбасына сәйкес келеді. Әзірлеуші негізгі дерек әрекеттерін Python кластары және сұрау құрылымдары арқылы орындайды. Дегенмен SQL, транзакция және дерекқор байланыстарын түсіну қажет болып қалады.

2. `category_id` пен `category` Relationship өрісінің айырмасы қандай?

`category_id` — `rooms` кестесінде сақталатын нақты сыртқы кілт бағаны. Ол `room_categories.id` мәніне сілтейді. `category` — тиісті санат объектісіне қатынауға арналған ORM байланысы; оның атымен қосымша SQL бағаны жасалмайды. Бірінші өріс кілт мәнін, екіншісі Python объектілері арасындағы қатынасты береді.

3. Seed деректер не үшін қажет? Бос базамен жұмыс неге ыңғайсыз?

Seed — жүйені алғашқы үлгі деректермен толтыру. Қонақтар, санаттар, нөмірлер мен брондаулар болғанда API тізімін, сыртқы кілттерді, сүзгілерді және қайшылықтарды бірден тексеруге болады. Бос база дұрыс сүзгі мен қате сүзгінің екеуіне де бос тізім қайтарып, мәселені жасыруы мүмкін. StayBook-та `python seed.py` бастапқы деректерді енгізеді, қайта іске қосу бірдей жазбаларды көбейтпейді.

4. `create_engine` ішіндегі `echo=True` қандай рөл атқарады?

Бұл параметр орындалатын SQL сұрауларын және транзакция логтарын консольге шығарады. Кесте жасалғанын, дұрыс `SELECT` немесе `INSERT` орындалғанын көруге көмектеседі. StayBook-та оны `SQL_ECHO=1` арқылы қосуға болады. Логты қарағанда енгізілген деректердің де көрсетілуі мүмкін екенін ескереміз.

5. Неліктен `database.db` Git-ке кірмейді, ал `models.py` міндетті түрде кіреді?

`models.py` деректер құрылымын анықтайтын бастапқы код, сондықтан өзгерістері бақыланады. `database.db` — жұмыс барысында өзгеретін жергілікті деректер файлы; ол көлемді болуы, жеке мәлімет қамтуы және біріктіру қайшылығын туғызуы мүмкін. Құрылым `init_db()`, ал мысал мазмұн `seed.py` арқылы қайта құрылады.

Зертханалық жұмыс №3

1. REST API деген не? GET пен POST айырмасы қандай?

REST API ресурстарға HTTP арқылы қатынауды ұйымдастырады. StayBook-та ресурстар — `/rooms`, `/guests`, `/bookings`. `GET` деректі оқиды және сервер күйін өзгертпеуі тиіс. `POST` көбіне жаңа ресурс жасайды; сұрау денесіне жаңа жазбаның деректері жіберіледі және сәтті жасағанда `201 Created` қайтады.

2. FastAPI ішіндегі Pydantic не істейді? Ол қате енгізуден қалай қорғайды?

Pydantic сұрау өрістерінің типтерін, шектерін және өзара шарттарын тексереді, сондай-ақ жауап құрылымын сипаттауға көмектеседі. Мысалы, теріс `price_per_night`, нөлдік `capacity` немесе бос `room_number` қабылданбайды. Схемаға сәйкес емес

сұрауға FastAPI `422` қайтарады. Бұл логин мен пайдаланушы құқықтарын тексеруді алмастырмайды.

3. `Depends` деген не? Неге барлық маршрутқа бір ортақ сессияны импорттамаймыз?

`Depends` — тәуелділікті енгізу механизмі. `Depends(get_session)` маршрутқа оның сұрауына арналған сессияны береді. Генератор сессияны ашып, өңдеуден кейін жабады. Бір ортақ ұзақ өмір сүретін `Session` параллель сұраулардың транзакцияларын араластырып, қате күй қалдыруы мүмкін. Ал `engine` объектісін ортақ пайдалану қалыпты: engine мен Session бір ұғым емес.

4. Swagger UI қалай жасалады? Оның HTML кодын қолмен жазу қажет пе?

FastAPI маршруттардан, типтерден және Pydantic схемаларынан OpenAPI спецификациясын жасайды. Дайын Swagger UI сол спецификацияны қолданып, интерактивті құжаттаманы көрсетеді. Қолмен HTML жазу қажет емес. StayBook үшін мекенжай — `http://127.0.0.1:8000/docs`, спецификация — `/openapi.json`.

5. `422 Unprocessable Entity` қашан қайтады?

Сұрау форматы түсінікті болғанымен, оның өрістері қабылдау схемасына сәйкес болмағанда `422` қайтады. Мысалы, теріс баға, жарамсыз күн, міндетті өрістің болмауы немесе міндетті өріске `null` жіберу. Бұл `404`-тен өзгеше: `404` дұрыс пішіндегі ID-мен ізделген ресурс табылмағанын білдіреді.

Зертханалық жұмыс №4

1. PUT пен PATCH айырмасы қандай? Қайсысы трафикті азайтуы мүмкін?

`PUT` ресурстың толық көрінісін ауыстыруға, `PATCH` оның бір бөлігін өзгертуге арналған. Нөмірдің тек бағасын өзгерту үшін PATCH сұрауымен бір өріс жіберу жеткілікті, сондықтан мұндай жағдайда трафик аз болады. Барлық жағдайда бір әдіс аз трафик береді деген ереже жоқ: ол жіберілетін дерек көлеміне байланысты.

2. Query параметрлері деген не және URL ішінде қалай беріледі?

Query параметрлері URL жолындағы `?` таңбасынан кейін жазылады, бірнешеуі `&` арқылы бөлінеді. Мысалы: `/rooms?search=10&min_price=10000&max_price=30000`. Олар ресурсқа қосымша іздеу не сүзгі шарттарын береді. Нөмірдің ID-і `/rooms/5` түрінде берілсе, ол query емес, path параметрі болады.

3. PATCH кезінде None мәндерінен қалай қорғанамыз?

`model_dump(exclude_unset=True)` клиент жібермеген өрістерді жаңарту сөздігінен алып тастайды. Бірақ клиент нақты `null` жіберген өріс бұл сөздікке кіреді. Сондықтан міндетті өрістерге `null` мәнін схема валидаторымен қабылдамаймыз; дерекқордың `NOT NULL` шектеуі қосымша қорғаныс береді. `Optional` жалғыз өзі бұл мәселені шешпейді. `exclude_none=True` қолдану барлық жағдайда балама емес: кейбір рұқсат етілген өрістерді клиент әдейі босатуы мүмкін.

4. Жою алдында ID бойынша жазбаны не үшін іздейміз?

Алдымен жазбаның бар-жоғын біліп, табылмаса түсінікті `404` қайтарамыз. Сонымен бірге бизнес шарттарын тексереміз: мысалы, брондауы бар нөмірді жоймау.

Базаның сыртқы кілттері мен транзакциялар да қажет; алдын ала іздеу дерекқор шектеулерін алмастырмайды.

5. CRUD деген не? Қандай HTTP әдістері сәйкес келеді?

Әрекет	Мағынасы	HTTP әдісі
Create	Жаңа жазба қосу	`POST`
Read	Тізімді немесе бір жазбаны оқу	`GET`
Update	Жазбаны толық немесе ішінара өзгерту	`PUT` немесе `PATCH`
Delete	Жазбаны жою	`DELETE`

StayBook-та ішінара жаңарту `PATCH` арқылы іске асырылады.

Нақты тексеру нәтижелері

Тексерілген күн: **2026-11-11 (UTC)**. Тексеру осы жобаның орындалатын кодында жүргізілді.

Автоматты тексеру

```
python -m pytest tests -q
32 passed in 2.80s
```

32 интеграциялық тексеру әр тестке бөлек уақытша SQLite базасын қолданды.

Бағыт	Нәтиже
Төрт ORM кестесі, сыртқы кілттер және Relationship	Өтті
Seed-ті жеке командамен екі рет орындау	Әр кестеде 5 жазба; қайталанған жазба жоқ
GET тізім және ID бойынша іздеу	Өтті; жоқ ID — 404
POST және деректерді валидациялау	Өтті; сәтті — 201, жарамсыз — 422
Бірегей нөмір және сыртқы кілтті тексеру	Өтті; қайталану — 409, жоқ байланыс — 404
PATCH кезінде тек берілген өрістерді өзгерту	Өтті; қалған өрістер сақталады, айқын null — 422
DELETE және жойылған ID-ге қайта сұрау	Өтті; жауап денесі жоқ 204, кейін 404
Іздеу, баға аралығы, санат және қате шектер	Өтті
Брондау мерзімдері және қабаттасу	Өтті; қайшылық — 409, қатарлас күндерге рұқсат
Брондауды болдырмау және қайта белсендіру	Өтті
Байланысқан нөмірді жоюдан қорғау	Өтті; 409

`pip check`: **No broken requirements found.** Барлық Python файлдарының синтаксисі тексерілді. Тәуелділіктің Starlette пакеті тест кезінде httpx интеграциясы туралы ескерту шығарды; тексерулер сәтті аяқталды.

Браузерде тексеру

Жергілікті FastAPI және Streamlit серверлері Chromium браузерінде ашылды. Скриншоттар нақты жұмыс істеп тұрған беттерден алынды:

- Swagger жалпы көрінісі.
- GET /rooms: 200 жауабы.
- Теріс баға: POST /rooms → 422.
- Нөмірлер интерфейсі.
- Брондаулар интерфейсі.

Streamlit-тің төрт бөлімі AppTest арқылы да ашылып тексерілді. Браузерде нөмірлер мен брондаулар беттері қосымша тексерілді. Интерфейстен жаңа нөмір қосу және растау белгісімен өшіру толық орындалды; кейін сол ID-ге API сұрауы 404 қайтарды. Тексеру үшін қосылған уақытша нөмір тазартылды.

Орындалу ортасы

Python 3.12.14, FastAPI 0.162.4, SQLModel 0.0.48, Streamlit 1.67.0, pytest 9.2.0. Орнатылған пакеттердің нақты нұсқалары `requirements.lock.txt` ішінде сақталған.

Бұл нәтижелер әзірлеу ортасындағы тексеруге жатады. Студенттің жеке компьютерінде Cursor/VS Code орнатылғаны немесе қорғауды өзі өткізгені туралы мәлімдеме жасалмайды. Жеке компьютерде іске қосу қадамдары README ішінде берілген.